2 Data Compression, Weak AEP and Lossless Source Coding

Reading: Chapter 3 of Cover and Thomas

Review

- Definitions of entropy, mutual information and divergence.
- Conditional entropy, conditional mutual information, and chain rules.
- Convexity/concavity of information measures.
- Data processing inequality and Fano's inequality.

Preview of Lec. 7–10

- Fixed-length source coding
- Variable-length source coding
- Optimal prefix-free code: Huffman code
- Weak AEP: law of large numbers applied to $-\log P_X(X)$
- Source coding theorem: how many information bits are required to compress a random source and reliably recover it?

2.1 Source coding for discrete sources: fixed-length source coding

In this section, we consider data compression (source coding problem). The source encoder converts the sequence of symbols from the source to a sequence of binary bits, preferably using as few binary bits per symbol as possible. The source decoder performs the inverse operation.

We mainly focus on source coding and decoding for discrete sources. The output of a discrete source is a sequence of symbols from a known discrete alphabet $\mathcal{X}$. This alphabet could be the alphanumeric characters, the characters on a computer keyboard, English letters, the symbols in sheet music, binary digits, etc.

The simplest approach to encoding a discrete source into binary bits is to create a code C that maps each symbol $x \in \mathcal{X}$ into a distinct codeword $C(x)$, where $C(x)$ is a block of binary digits.

Definition 1 (Fixed-length code)**.** *When each codeword $C(x)$ is restricted to have the same block length L, the code is called a fixed-length code.*

Example: If the alphabet $\mathcal{X}$ consists of the 7 symbols $\{a, b, c, d, e, f, g\}$, the the following fixed-length code of block length $L = 3$ could be used.

$$\begin{aligned} C(a) &= 000 \\ C(b) &= 001 \\ C(c) &= 010 \\ C(d) &= 011 \\ C(e) &= 100 \\ C(f) &= 101 \\ C(g) &= 110. \end{aligned}$$

The source output $x_1, x_2, \ldots$ would then be encoded into the encoded output $C(x_1)C(x_2)\ldots$ and thus the encoded output contains L bits/source symbol. If the source alphabet has size $|\mathcal{X}| = M$, then this encoding method requires $L = \lceil \log_2 M \rceil$ bits to encode each source. Thus

$$\log_2 M \leq L < \log_2 M + 1. \tag{1}$$

When M is a power of 2, the lower bound, $L = \log_2 M$, can be achieved. But otherwise, L should be strictly larger than $\log_2 M$. Is there any way we can reduce L?

A simple technique is to compress length-n sequence of symbols $x_1, x_2, \ldots, x_n$ together. Given an alphabet $\mathcal{X}$ of $|\mathcal{X}| = M$, there are M^n possible length-n sequences. Using fixed-length source coding on these length-n sequence, each sequence can be encoded into $L = \lceil \log_2 M^n \rceil$ bits. When we define the rate of source coding $R = L/n$ (bits/source), this approach achieves

$$\log_2 M = \frac{n \log_2 M}{n} \leq R = \frac{\lceil \log_2 M^n \rceil}{n} < \frac{n \log_2 M + 1}{n} = \log_2 M + \frac{1}{n}. \tag{2}$$

By letting $n \to \infty$, the average number of bits per source symbol can be made arbitrarily close to $\log_2 M$, regardless of whether M is a power of 2.

Remarks:

- This result begins to hint at why measures of information are logarithmic in the alphabet size. Moreover, this is the first result where we can observe the benefit of compressing a large number of sources togehter, i.e., in asymptotic regimes ($n \to \infty$).
- This method is nonprobabilistic; it takes no account of whether some symbols occur more frequently than others. But if it is known that some symbols occur more frequently than others, then the rate R of coded bits per source symbol can be reduced by assigning shorter bit sequences to more common symbols in a *variable-length source code.* This will be our next topic.

2.2 Variable-length codes for discrete sources

Definition 2 (Discrete Memoryless Source)**.** *A sequence $\{X_i\}_{i=1}^{\infty} \sim$ i.i.d. P on $\mathcal{X}$ is called a DMS(P).*

Definition 3 (Variable-length code)**.** *A variable-length code C maps each source symbol x in a source alphabet $\mathcal{X}$ to a binary string $C(x)$, called a codeword. The number of bits in $C(x)$ is called the length $l(x)$ of $C(x)$. The rate R (the average number of bits per source symbol) for the variable-length code is defined by*

$$R = \sum_{x \in \mathcal{X}} l(x) p(x). \tag{3}$$

Example: Consider a variable-length code for the alphabet $\mathcal{X} = \{a, b, c\}$ such that

$$\begin{aligned} C(a) &= 0 \qquad & l(a) &= 1 \\ C(b) &= 10 \qquad & l(b) &= 2 \\ C(c) &= 11 \qquad & l(c) &= 2 \end{aligned}$$

Successive codewords of a variable-length code are assumed to be transmitted as a continuing sequence of bits, with no demarcations of codeword boundaries (i.e., no commas or spaces). The source decoder, given an original starting point, must determine where the codeword boundaries are; this is called *parsing.*

The major property that is usually required from any variable-length code is that of unique decodability. This essentially means that for any sequence of source symbols, that sequence can be reconstructed unambiguously from the encoded bit sequence.

Definition 4 (Unique decodability)**.** *A code C for a discrete source is uniquely decodable if, for any string of source symbols, say $x_1, x_2, \ldots, x_n$, the concatenation of the corresponding codewords, $C(x_1)C(x_2)\ldots C(x_n)$, differs from the concatenation of the codewords $C(x'_1)C(x'_2)\ldots C(x'_m)$ for any other string $x'_1, x'_2, \ldots, x'_m$ of source symbols.*

For example, the above code C for the alphabet $\mathcal{X} = \{a, b, c\}$ is soon shown to be uniquely decodable. However, the code C' defined by

$$C(a) = 0$$
$$C(b) = 1$$
$$C(c) = 01$$

is not uniquely decodable, even though the codewords are all different. if the source decoder observes 01, it cannot determine whether the source emitted (ab) or (c).

Decoding the output from a uniquely-decodable code, and even determining whether it is uniquely decodable, can be quite complicated. However, there is a simple class of uniquely-decodable codes called *prefix-free codes*.

Definition 5 (Prefix-free code)**.** *A prefix of a string $y_1 \dots y_l$ is any initial substring $y_1 \dots y_{l'}$, $l' \leq l$ of that string. The prefix is proper if $l' < l$. A code is prefix-free if no codeword is a prefix of any other codeword.*

For example, the code C with codewords 0, 10 and 11 is prefix-free, but the code C' with codewords 0, 1 and 01 is not. Every fixed-length code with distinct codeword is prefix-free.

We will now show that every prefix-free code is uniquely decodable. The proof is constructive, and shows how the decoder can uniquely determine the codeword boundaries.

Given a prefix-free code C, a corresponding *binary code tree* can be defined which grows from a root on the left to leaves on the right representing codewords. Every branch is labelled 0 or 1 and each node represents the binary string corresponding to the branch labels from the root to the node. The tree is extended just enough to include each codeword. That is, each node in the tree is either a codeword or proper prefix of a codeword.

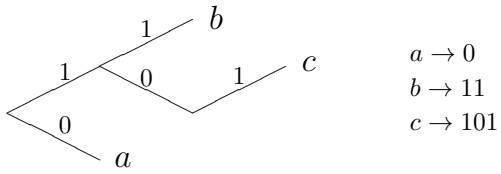

Figure 1: The binary code tree for a prefix-free code

The prefix-free condition ensures that each codeword corresponds to a leaf node (i.e., a node with no adjoining branches going to the right). Each intermediate node (i.e., nodes having one or more adjoining branches going to the right) is a prefix of some codeword reached by traveling right from the intermediate node. The tree of Figure 1 has an intermediate node, 10, with only one right-going branch. This shows that the codeword for c could be shortened to 10 without destroying the prefix-free property. This is shown in Figure 2. A prefix-free code will be called

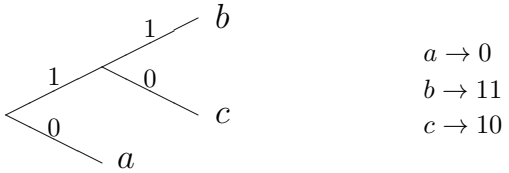

Figure 2: A code with shorter lengths than that of Fig. 1

full if no new codeword can be added without destroying the prefix-free property. As just seen,

a prefix-free code is also full if no codeword can be shortened without destroying the prefix-free property. Thus the code of Figure 1 is not full, but that of Figure 2 is.

A way to see that prefix-free codes are uniquely decodable is to look at the codeword parsing problem from the viewpoint of the source decoder. Given the encoded binary string for any string of source symbols, the source decoder can decode the first symbol simply by reading the string from left to right and following the corresponding path in the code tree until it reaches a leaf, which must correspond to the first codeword by the prefix-free property. After stripping off the first codeword, the remaining binary string is again a string of codewords, so the source decoder can find the second codeword in the same way, and so on.

For example, suppose a source decoder for the code of Figure 2 decodes the sequence 1010011... . Proceeding through the tree from the left, it finds that 1 is not a codeword, but that 10 is the codeword for c. Thus c is decoded as the first symbol of the source output, leaving the string 10011.... Then c is decoded as the next symbol, leaving 011..., which is decoded into a and then b, and so forth.

It has been shown that all prefix-free codes are uniquely decodable. The converse is not true, as shown by the following code:

$$C(a) = 0$$
$$C(b) = 01$$
$$C(c) = 011$$

An encoded sequence for this code can be uniquely parsed by recognizing 0 as the beginning of each new code word.

How can we construct the minimum average-length (minimum rate) prefix-free code? This is our next topic.

2.3 Huffman's algorithm for optimal source coding

David Huffman (1950) came up with a very simple and straightforward algorithm for constructing optimal (in the sense of minimum rate R (bits/source)) prefix-free codes by discovering a few simple properties the optimal code should satisfy.

Lemma 1. *Optimal codes have the property that if $p_i > p_j$, then $l_i \leq l_j$.*

Lemma 2. *Optimal prefix-free codes have the property that the associated code tree is full.*

Lemma 3. *Let X be a random symbol with a pmf satisfying $p_1 \geq p_2 \geq \cdots \geq p_M$. There is an optimal prefix-free code for X in which the codewords for $M-1$ and M are siblings and have maximal length within the code, i.e., $l(1) \geq l(2) \cdots \geq l(M-1) = l(M)$.*

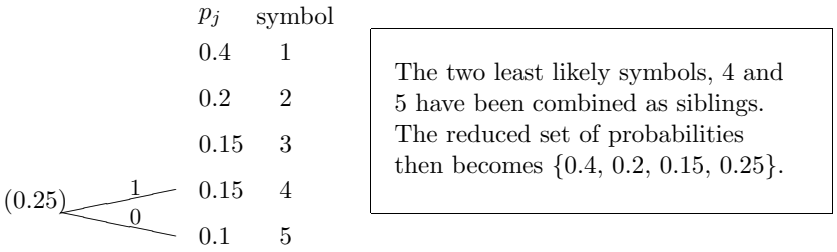

Figure 3: Step 1 of the Huffman algorithm.

The Huffman algorithm works in the following way.

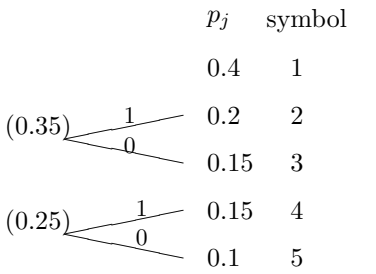

Figure 4: Step 2 of the Huffman algorithm.

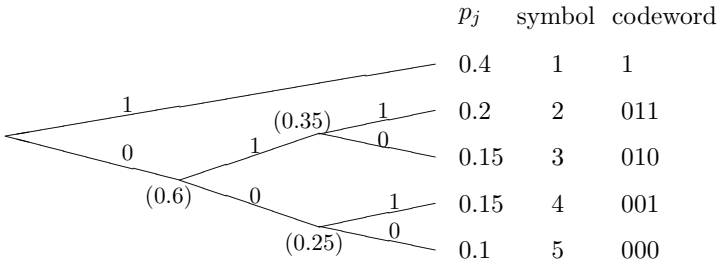

Figure 5: The completed Huffman code.

- Step1: Arrange the probabilities $[p_1, p_2, \ldots, p_M]$ as decreasing order. Choose the two least likely symbols, specially M and $M-1$, and constraining them to be siblings in the yet unknown code tree. See Fig. 3.
- Step2: If the above pair of siblings is removed from the yet unknown tree, the rest of the tree must contain $M-1$ leaves, namely $M-2$ leaves for the original first $M-2$ symbols and the parent node of the removed siblings. Define $p'_{M-1} = p_{M-1} + p_M$ and consider a new pdf $[p_1, p_2, \ldots, p_{M-2}, p'_{M-1}]$ for a new random variable X' of $M-1$ symbols. We next construct an optimal code for X' by arranging the probabilities as decreasing order and again choosing the two least likely symbols and letting them be siblings in the yet unknown code tree. See Fig. 4.
- Repeat the above process until the completion of the tree. See Fig. 5.

We next show the optimality of Huffman code.

Lemma 4. *Huffman algorithm constructs the optimal prefix-free code (in the sense of minimum R).*

Proof. As shown in Lemma 3, for an optimal prefix-free code for X the codewords for $M-1$ and M are siblings and have maximal length within the code. For all codes for X in which $C(M-1)$ and $C(M)$ are siblings, the expected length R' (bits/source symbol) of X' and the corresponding R of the code for X are related as

$$R = R' + p_{M-1} + p_M. \tag{4}$$

This shows that R is minimized by minimizing R', and $R_{\mathsf{min}} = R'_{\mathsf{min}} + p_{M-1} + p_M$. The way Huffman algorithm constructs the code indeed minimizes R' of X' again by Lemma 3. Therefore, the Huffman algorithm constructs the minimum average-length prefix-free code. □

Remarks:

- Huffman code constructs the optimal prefix-free code. But neither the Huffman algorithm nor its proof of optimality give any hint on how the rate R depends on the distribution P of the source X. Huffman code is just an optimal algorithm. It does not answer for more fundamental questions such as if I change the source how many more/less bits would be required for compression.
- Below, we learn some tools to discuss about such fundamental limits on data compression.

References

Gallager (2008), *Principles of Digital Communication*, Cambridge University Press.

2.4 Weak Asymptotic Equipartition Property (AEP)

In the next lecture, we will talk about the fundamental limits of source coding (the minimum number of bits required to compress a random source). What is source coding? In general, source coding is a mapping of whatever source, e.g., image, speech, or video, into bit streams. The goal is to use as few bits as possible, while allowing the source to be reconstructed from these bits. To learn how to do source coding, in this lecture, we learn a very useful mathematical tool called AEP. Let's start from reviewing weak law of large numbers.

Weak Law of Large Numbers (WLLN)

Let $X_1, X_2, \ldots$ be i.i.d. with mean μ and variance $\sigma^2 < \infty$. Let

$$S_n \triangleq \frac{1}{n}[X_1 + X_2 + \cdots + X_n].$$

Theorem 1 (WLLN)**.**

$$S_n \xrightarrow{P} \mu, \quad , i.e., \quad (S_n - \mu) \xrightarrow{P} 0. \tag{5}$$

In other words, for all $\epsilon, \delta > 0$, there exists N_0 such that $\forall n > N_0$,

$$\Pr(|S_n - \mu| > \delta) < \epsilon, \tag{6}$$

i.e., for all $\delta > 0$,

$$\lim_{n\to\infty} \Pr[|S_n - \mu| > \delta] = 0. \tag{7}$$

Proof. Markov inequality: If $Z \geq 0$ and $\gamma > 0$, then $\Pr(Z > \gamma) < \frac{\mathbb{E}[Z]}{\gamma}$
Proof of Markov inequality:

$$\mathbb{E}[Z] = \underbrace{\mathbb{E}[Z|Z > \gamma]}_{>\gamma} \Pr(Z > \gamma) + \underbrace{\mathbb{E}[Z|Z \leq \gamma] \Pr(Z \leq \gamma)}_{\geq 0}. \tag{8}$$

Let $Z = |S_n - \mu|^2$. Then $\mathbb{E}[Z] = \frac{\sigma^2}{n}$, which goes to 0 as n increases. By using Markov inequality,

$$\Pr(|S_n - \mu|^2 > \delta^2) < \frac{\sigma^2}{n\delta^2}. \tag{9}$$

□

Asymptotic Equipartition Property (AEP)

Let $X_1, X_2, \ldots, X_n$ be i.i.d. with P over $\mathcal{X}$. Consider $Y_i = -\log P(X_i)$. Then, $\mathbb{E}[Y_i] = H(P)$. Let us denote $(x_1, x_2, \ldots, x_n)$ by x_1^n or simply by x^n.

Theorem 2. *By applying WLLN,* $\forall \epsilon, \delta > 0$, $\exists N_0$ *such that* $\forall n > N_0(\epsilon, \delta)$,

$$P^n \left[\left\{ x_1^n : \left| -\frac{1}{n} \sum_{i=1}^{n} \log P(x_i) - H(P) \right| > \delta \right\} \right] < \epsilon. \tag{10}$$

Remarks:

- Notation: $P^n(\cdot)$ is the product of n identical and independent distributions, each of which is P.
- The above inequality holds since Y_i has a constant variance, i.e., $\text{var}(-\log P(X)) < \infty$.
- Since $\sum_{i=1}^{n} \log P(X_i) = \log P^n(X_1^n)$, the inequality (10) is equivalent to

$$P^n \left[\left\{ x_1^n : \left| -\frac{1}{n} \log P^n(x_1^n) - H(P) \right| > \delta \right\} \right] < \epsilon. \tag{11}$$

- This theorem says that for large enough n, with high probability, X^n falls in a particular set, where all x_1^n has somewhat similar probabilities under the distribution P^n. This motivates the following definition.

Definition 6 (Weak typical set)**.**

$$A_\delta^{(n)}(P) \triangleq \left\{ x_1^n \in \mathcal{X}^n : \left| -\frac{1}{n} \log P^n(x_1^n) - H(P) \right| \leq \delta \right\}. \tag{12}$$

Corollary 1. *Theorem 2 implies that*

$$P^n(A_\delta^{(n)}(P)) \geq 1 - \epsilon. \tag{13}$$

Remarks:

- To reduce the number of parameters, we consider the case $\delta = \epsilon$. Theorem 2 implies

$$P^n(A_\epsilon^{(n)}(P)) \geq 1 - \epsilon. \tag{14}$$

- When we generate a sequence $X_1^n \in \mathcal{X}^n$ from the distribution P^n, with high probability, this sequence falls in the weak typical set $A_\epsilon^{(n)}(P)$. The set $A_\epsilon^{(n)}(P)$ takes almost all the probability. Notice that $A_\epsilon^{(n)}(P)$ is just a subset of $\mathcal{X}^n$.
- The weak typical set can be written as

$$\begin{aligned} A_\epsilon^{(n)}(P) &\triangleq \left\{ x_1^n \in \mathcal{X}^n : \left| -\frac{1}{n} \log P^n(x_1^n) - H(P) \right| \leq \epsilon \right\} \\ &= \left\{ x_1^n : 2^{-n(H(P)+\epsilon)} \leq P^n(x_1^n) \leq 2^{-n(H(P)-\epsilon)} \right\}. \end{aligned} \tag{15}$$

- All sequences in the weak typical set are approximately equiprobable, i.e., $P^n(x_1^n) \approx 2^{-nH(P)}$. Note that $nH(P)$ is the average surprise we get when we observe an arbitrary X_1^n.

In the following, we check the size of the typical set $A_\epsilon^{(n)}$.

Corollary 2 (Size of the typical set)**.** *1)* $|A_\epsilon^{(n)}(P)| \leq 2^{n(H(P)+\epsilon)}$.
2) For large enough n, $|A_\epsilon^{(n)}(P)| \geq (1-\epsilon)2^{n(H(P)-\epsilon)}$.

Proof. From the definition of $A_\epsilon^{(n)}(P)$, $\forall x^n \in A_\epsilon^{(n)}(P)$, we have

$$2^{-n(H(P)+\epsilon)} \leq P^n(x_1^n) \leq 2^{-n(H(P)-\epsilon)}. \tag{16}$$

Therefore,

$$|A_\epsilon^{(n)}(P)|2^{-n(H(P)+\epsilon)} \leq P^n(A_\epsilon^{(n)}(P)) \leq |A_\epsilon^{(n)}(P)|2^{-n(H(P)-\epsilon)}. \tag{17}$$

Since $P^n(A_\epsilon^{(n)}(P)) \leq 1$, the fact 1) follows. For large enough n, since $\Pr(A_\epsilon^{(n)}(P)) \geq 1-\epsilon$ the fact 2) follows. □

Remarks:

- The first statement in this corollary does not require large enough n, but the second statement does.
- The size of $A_\epsilon^{(n)}(P) \approx 2^{nH(P)}$ is exponentially smaller than $|\mathcal{X}|^n = 2^{n\log|\mathcal{X}|}$.

Exponential Approximation

This will allow us to focus on exponentially increasing or decreasing rate of sequences as $n \to \infty$.

Definition 7. *For a sequence* $f(1), f(2), \ldots f(n)$*, and a constant* $c \in \mathbb{R}$*, we say*

$$f(n) \doteq e^{cn}, \quad \textit{iff}, \quad \lim_{n\to\infty} \frac{\log f(n)}{n} = c. \tag{18}$$

Furthermore, we say

$$f(n) \doteq h(n) \tag{19}$$

if both have the same exponential rate, i.e., $f(n) \doteq h(n) \doteq e^{cn}$.

Example:

- $f(n) \doteq 2f(n) \doteq 1000f(n)$
- $f(n) \doteq n^2 f(n)$
- $f(n) \doteq e^{1000n^{0.99}} f(n)$

Exercise: True or False

$$f(n) \doteq f(n) + e^{-n} \tag{20}$$

Remarks:

- In this notation, we use the base e for the log.
- For ϵ_n approaching 0 as $n \to \infty$, when $\frac{\log f(n)}{n} \in [c-\epsilon_n, c+\epsilon_n]$, we can write $f(n) \doteq e^{cn}$.
- By Markov inequality,

$$P^n\left[\left|-\frac{1}{n}\sum_{i=1}^n \log P(X_i) - H(P)\right| > \delta_n\right] \leq \frac{\mathrm{var}(-\log P(X))}{n\delta_n^2} \tag{21}$$

By choosing δ_n that decreases slower than $\frac{1}{\sqrt{n}}$, the above probability goes to 0 as $n \to \infty$, i.e., $\epsilon_n := \frac{\mathrm{var}(-\log P(X))}{n\delta_n^2}$ goes to 0. Therefore, for such δ_n,

$$P^n(A_{\delta_n}^{(n)}) \to 1. \tag{22}$$

Under this exponentially approximation, AEP can be rewritten as follows.

- $\forall x_1^n \in A_{\delta_n}^{(n)}(P)$, $P^n(x^n) \doteq 2^{-nH(P)}$.
- The size of set equals $|A_{\delta_n}^{(n)}(P)| \doteq 2^{nH(P)}$.
- $P^n(A_{\delta_n}^{(n)}) \to 1$

Now we have a nice picture of AEP. Suppose we have a length-n sequence X^n whose symbols are i.i.d. with distribution P over a finite alphabet $|\mathcal{X}|$. The total number of possible realizations of X^n is $|\mathcal{X}|^n = 2^{n \log |\mathcal{X}|}$. With AEP, we know that with a high probability X^n falls in $A_\epsilon^n(P)$. Every sequence in the set $A_\epsilon^n(P)$ is approximately equiprobable with probability $2^{-nH(P)}$. Moreover, the size of the set $A_\epsilon^n(P)$ is $|A_\epsilon^n(P)| \doteq 2^{nH(P)}$, which is "exponentially small" than the entire set $|\mathcal{X}|^n$. We next use these nice facts to design source coding.

2.5 Lossless source coding

In this lecture, we consider a source coding for discrete memoryless source. Remind the definition of DMS: A sequence $\{X_i\}_{i=1}^\infty \sim$ i.i.d. P on $\mathcal{X}$ is called a DMS(P). **Source Coding**

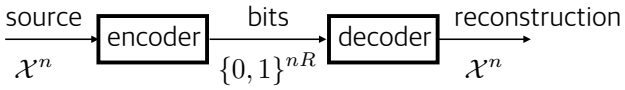

Figure 6: Encode n symbols together. Try to use as few bits as possible

Definition 8 (Souce Code)**.** *An (n, R) block source code for a DMS(P) is a pair of maps, f_n (encoder) and g_n (decoder):*

$$\begin{aligned} f_n &: \mathcal{X}^n \to \{0,1\}^{nR} = \{1, 2, \ldots, 2^{nR}\} \\ g_n &: \{0,1\}^{nR} \to \mathcal{X}^n. \end{aligned} \tag{23}$$

The probability of error for this source code is defined as

$$P_e^{(n)} \triangleq P^n(\{x^n : g_n(f_n(x^n)) \neq x^n\}). \tag{24}$$

Remarks:

- If f_n is a one-to-one mapping, R should satisfy $nR \geq \lceil \log |\mathcal{X}|^n \rceil$. (When n large, integer effect goes away, and $R \geq \log |\mathcal{X}|$.)

Definition 9. *For any $\epsilon > 0$, we say a rate R is an achievable rate of source coding for DMS(P) with probability of error ϵ, if there exists a sequence of (n, R) codes such that*

$$\limsup_{n \to \infty} P_e^{(n)} \leq \epsilon. \tag{25}$$

The minimum of such rates for a given ϵ is denoted as $R_{\min}(\epsilon)$, which is the achievable source coding rate allowing error probability ϵ. We further define

$$R_{\min} = \lim_{\epsilon \to 0} R_{\min}(\epsilon). \tag{26}$$

Theorem 3 (Lossless Source Coding Theorem)**.** *The minimum achievable rate of lossless source coding for DMS(P) is*

$$R_{\min} = H(P). \tag{27}$$

Proof of Source Coding Theorem. The proof of every information theoretic theorem always contain two parts. The achievable part to show that certain performance can be achieved, and the converse part to show that no scheme can achieve the performance beyond certain limit. Hopefully, the two parts would match, like in this theorem.
What to show:

1) Achievability: if $R > R_{\min} = H(P)$, there exists a sequence of (n, R) codes such that $\lim_{n\to\infty} P_e^{(n)} = 0$.
2) Converse: if $R < R_{\min} = H(P)$, there is no code guaranteeing $\lim_{n\to\infty} P_e^{(n)} = 0$.

Achievability:
One can enumerate only the sequences in the typical set of size $|A_\delta^{(n)}(P)| \le 2^{n(H(P)+\delta)}$. For any $R > H(P)$, one can pick δ such that $R > H(P) + \delta > H(P)$. From WLLN, $P_e^{(n)} = P^n(X^n \notin A_\delta^{(n)}) \to 0$ as $n \to \infty$.

Converse:
We will use Fano's inequality to prove the converse. Fano's inequality focuses on the relation between $H(X^n|Y)$ and $P_e^{(n)} = \Pr(g_n(Y) \neq X^n)$ where $Y = f_n(X^n)$ is the codeword (bit sequence). More specifically, Fano's inequality says that after observing Y if we still have some uncertainty about X^n, which is quantified by $H(X^n|Y)$, $P_e^{(n)}$ cannot be too small.

$$\begin{aligned} nH(P) &= H(X^n) \\ &= H(X^n|Y) + I(X^n;Y) \\ &\le H(X^n|Y) + H(Y) \\ &\le H(X^n|Y) + nR \\ &\le H(\mathbb{1}_E) + P_e^{(n)} \log(|\mathcal{X}|^n - 1) + nR \\ &\le (nR + 1) + P_e^{(n)} \log |\mathcal{X}|^n \end{aligned} \tag{28}$$

By rearranging terms, we have

$$P_e^{(n)} \ge \frac{H(P) - R - \frac{1}{n}}{\log |\mathcal{X}|}. \tag{29}$$

If $R > H(P)$, this is useless inequality. If $R < H(P)$, $P_e^{(n)}$ is bounded below by a positive constant ($\lim_{n\to\infty} P_e^{(n)} \neq 0$). Another way to look at the inequality is

$$R \ge H(P) - P_e^{(n)} \log |\mathcal{X}| - 1/n. \tag{30}$$

As $n \to \infty$ and making $P_e^{(n)} \to 0$, we have $R \ge H(P)$ as a necessary condition of lossless source coding. □

Remarks:

1) Without the assumption of a probabilistic structure, the best thing one can do is to encode every possible sequence by a code of rate $R = \log |\mathcal{X}|$. Recall that this is the entropy of the uniform distribution over $\mathcal{X}$. For non-uniformly distributed source, a strictly lower rate, $R = H(P)$, would suffice for lossless source codes. The key reasons that we can do this are a) we assume a DMS (i.i.d. sequence) and b) we allow a small probability of error.
2) Here we considered a fixed-length source code: (f_n, g_n) formulation with codewords of nR bits. Alternative formulations of source coding would be variable-length source coding. One of such a variable length coding is Huffman code. Intuitively, if a particular outcome of the source has a smaller probability, one would want to encode it with a longer bit string,

to save the average number of bits. Let the length of a particular source $l(x) \approx -\log P(x)$. Note that for $x^n \in A_\delta^{(n)}$, $l(x^n) \approx -\log P^n(x^n) \approx n(H(P) \pm \delta)$. Therefore, when we focus on the typical set, fixed length code is not that bad.

3) Clearly, when an encoding error occurs for the fixed-length block coding, one can always employ a different mechanism to indicate error and resort to a difference coding scheme. For example, one can use a single bit in front of the codeword to indicate an error. The first bit equals 0 if the sequence is in the typical set and it equals 1 if the sequence is not in the typical set. For the case of the first bit equal to 0, the next nR bits indicate the corresponding codeword. For the first bit equal to 1, the next $n \log |\mathcal{X}|$ bits indicate the corresponding codeword. Note that this scheme doesn't hurt the overall rate (average number of bits per symbol), since the probability that a sequence of random source is not included in the typical set is arbitrary small as $n \to \infty$. Moreover, we can encode the source without any error.
4) The most interesting thing of this theorem might be the notion of "sequence of codes." This is a standard approach to allow asymptotic analysis. We will see many of such formulations in this course.